☄️ Velvet Comet · One-Pager (Nikhil Bindal)

🛠️ What I built

A completeness-first research engine on Firecrawl, exposed through a CLI, an HTTP API with live SSE, and a Next.js console. It targets the highest-ARR account in the customer feedback: a competitive-intel platform whose *"completeness is the product"* and whose deprecated deep-research workflow left a genuine gap.

The bet, in one line: **completeness is a coverage problem, not a limit problem.** Raising the limit returns more of the SEO head; the trade pubs, regional press, and niche forums analysts miss don't rank at any limit. So the engine varies the *queries*, not the count. It expands one query into angled sub-queries, fans them out through a rate-limited Firecrawl chokepoint, dedupes the head (SimHash plus a per-domain quota), and keeps going until a full round surfaces zero new domains. Then it reranks by intent and, on request, scrapes the top results for full content. Every run emits a coverage report: the saturation curve, source classes hit versus missed, honest gaps, and credits spent.

It composes the search and scrape endpoints, and it's durable: per-round checkpointing gives crash-resume with no re-spend, backed by either in-memory or Postgres behind one port. It runs hot and cold lanes and includes a Temporal port. 68 tests, 9 packages, strict-typed and lint-clean. **Validated live:** the gated contract tests pass against the real search endpoint, and on a real query, measured against the strongest flat baseline Firecrawl allows (one bare query at limit 100, which itself returned 78 domains), the engine still surfaced **21 source domains the flat search missed, 12 of them long-tail.**

⚖️ Key decisions and trade-offs

A pure, dependency-injected core, with all I/O behind ports. The completeness algorithm is a pure function over an injected clock, search port, and LLM port, with no network and no globals. The alternative, calling the Firecrawl SDK inline in each handler, ships faster but couples business logic to one runtime and makes the saturation loop untestable without live calls. The cost is real: more interfaces, and a composition root per surface. It paid for itself the moment the same engine ran unchanged in the CLI, the API coordinator, and the Temporal worker, and because the merge and saturation logic is verified by millisecond unit tests instead of paid batch runs.

Deterministic round-control, with the LLM only at the leaves. When to stop, how to merge, and what to keep is plain code; the LLM only generates sub-queries and scores results. The obvious alternative is a tool-using agent that decides its own next searches. I rejected it on purpose. This runs thousands of times a night against a credit budget, so it has to be reproducible, cost-bounded, and debuggable, and an agent is none of those (Firecrawl's own FIRE-1 agent carries non-deterministic credit cost, which is the tell). The trade-off is a less adaptive expansion strategy; the LLM seam is there to make it adaptive later if the eval justifies it.

I wrote the Firecrawl client by hand, behind a single chokepoint. A thin fetch-based client with the rate-limiter, leased semaphore, circuit breaker, cache, and cost ledger wrapped around it, rather than the SDK. The system is bottlenecked entirely on Firecrawl's per-plan rate and concurrency limits, so that budget has to be owned in exactly one place; an SDK hides retry and backoff and gives no shared view of spend. The trade-off is that I reimplement things the SDK gives for free and track the API's response shape myself, which is precisely why the contract tests exist, and why I can explain every line of the integration in the call.

Hand-rolled durability now, Temporal-portable by design. Per-round checkpointing in a small state machine (a serializable engine-state snapshot plus a pure step), so a crash resumes from the last completed round instead of re-spending credits. I didn't reach for Temporal on day one; the first cut shouldn't require standing up a cluster. But I shaped the state machine so it ports cleanly, then proved it by writing the

Temporal workflow, where durability is event-sourced and effectively free. The cost is two durability paths to keep honest; the payoff is evidence the design generalizes rather than being bespoke to one runtime.

Measured completeness, with the metric boundary stated honestly. "Completeness is the product" is unfalsifiable without a number, so I built an eval harness. A synthetic corpus with known ground truth yields a recall figure (+67% over a flat search), and a live script reports novel-domain coverage on real queries. I was deliberate about the line between them. Recall needs the *complete* set of correct sources, which doesn't exist on the open web, so live results are reported as novel domains versus the strongest flat baseline, never as recall. The synthetic number proves the mechanism; the live number proves it does something real on traffic. Eyeballing a few queries would have proven neither, and would have been the thing I'd most distrust from someone else.

Failures are values, and "partial" is never dressed up as "done." Expected failures are typed results with a retryable flag, and one taxonomy drives the retry logic, the circuit breaker, and the gap reporter. A run that can't reach some sources returns a partial result with the gaps recorded, never a fake-green "complete." For an analyst whose deliverable goes to their own client, a silent miss is worse than an honest gap. The trade is more verbose call sites than throw and catch.

🚫 What I deliberately did not build, and why

- **LinkedIn at scale (#10).** Policy-blocked by Firecrawl, with ToS and legal risk. The wrong thing to demo to an employer.
- **Asks that already ship.** Several requests are already config, not missing features, so I verified the product before building and declined to rebuild: fast snippet-only search (#4, covered by omitting scrape options), BYO and auto-retry proxies (#2, covered by proxy auto), and authenticated sessions (#11, covered by the profile and interact primitives). The right answer to those customers is "that's config, here's the parameter," not a feature.
- **Per-step action debugging (#7) and self-maintaining extractors (#9).** Both genuinely good, but each is its own deep product. Bundling them would have made this wide and shallow when the value is in going deep on one problem. They're the strongest next bets, noted as such.
- **Production ops I named but stubbed.** Multi-tenant auth and billing, worker autoscaling, a distributed rate limiter. The *interfaces* exist (the job store already has in-memory and Postgres implementations); the operational work is scoped out of a focused build, not hand-waved.

Narrow and deep: one customer's problem, solved end to end, durable and demoable.

🪲 What my AI tools got wrong, and how I caught it

The one worth telling was a test, not the product, and the lesson was about judgment, not syntax.

I had the model scaffold the unit tests for the diversity-merge step. It produced a fixture factory that generated mock results with near-identical default content: the plausible, repetitive boilerplate codegen loves to emit, like sequential titles and shared filler text. My SimHash near-duplicate collapser did exactly its job and merged them, so the per-domain-quota test failed. It asserted two results from a domain and got one.

The trap is the obvious fix. At that point the fast path to green is to relax the assertion or nudge the dedup threshold, and both are wrong. The test was correct: identical content *should* collapse. The fixture was lying about the scenario. The real fix was making the generator emit genuinely distinct content, so "distinct" inputs stay distinct and the dedup behavior stays pinned by the tests that actually exercise it.

The generalizable takeaway: **AI-generated test data is optimized to look reasonable, not to be representative.** It will cheerfully hand you inputs that are valid but degenerate in precisely the dimension

under test, and the failure then presents as a red test on *correct* code, which is the most expensive kind to misread. I caught it in seconds rather than in a paid batch run for one structural reason: the merge logic is a pure function in the core with no I/O, so it's exercised by deterministic millisecond tests. That same skepticism is why the live contract tests exist in the first place. Earlier, an agent surveying Firecrawl's API had confidently told me the hosted search endpoint accepts a custom-proxy parameter (it doesn't) and invented a deprecation date. I treat AI research as a draft to be checked against primary sources, and I encoded that distrust as tests that fail the build if the real API ever drifts from the schema the code assumes.